Solving 2048: Classical RL vs. LLM-Guided RLVR

A Comparative Study of Seven Reinforcement Learning Agents

Shriman Raghav Gautham Ramkumar
MS Robotics, Khoury College of Computer Sciences
Northeastern University, Boston, MA
srinivasan.shrim@northeastern.edu

I. Introduction

The puzzle 2048 is a compelling RL benchmark combining stochastic dynamics, a combinatorial state space ($\sim 10^{19}$ configurations), and sparse, delayed rewards. We conduct the first controlled comparison of six classical RL agents and an LLM fine-tuned with Group Relative Policy Optimization (GRPO), a form of Reinforcement Learning with Verifiable Rewards (RLVR).

Research questions. (Q1) How do five modern RL algorithms compare when architectural confounds are removed? **(Q2)** Can a 0.5B-parameter LLM learn 2048 strategy through GRPO alone? **(Q3)** How does sample efficiency scale from 1M to 5M training steps?

II. MDP Formulation

We formalize 2048 as an MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$:

A. State Space $\mathcal{S}$

The raw state is a $4{\times}4$ board $B \in \mathbb{Z}_{\geq 0}^{4\times 4}$, where each cell holds a tile value $\in \{0, 2, 4, ..., 2^{15}\}$. For CNN agents, we encode it as a 16-channel binary tensor $\mathbf{x} \in \{0,1\}^{16\times 4\times 4}$:

$$x_{k,i,j} = \mathbb{1}\left[B_{i,j} = 2^k\right], \quad k = 0, 1, \ldots, 15 \tag{1}$$

Channel 0 marks empty cells; channel k marks tiles of value 2^k, avoiding magnitude bias in convolutions. For the LLM agent, B is serialized as a labeled ASCII grid.

B. Action Space $\mathcal{A}$

$\mathcal{A} = \{0, 1, 2, 3\}$ mapping to $\{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}\}$.

C. Transition $\mathcal{T}(s' \mid s, a)$

Each transition has a deterministic component (slide & merge tiles) and a stochastic component (spawn a 2-tile with $p{=}0.9$ or 4-tile with $p{=}0.1$ at a random empty cell). If action a produces no board change, it is *invalid*—no tile spawns. The game terminates when no action changes the board.

D. Reward $\mathcal{R}(s, a, s')$

$$r = \begin{cases} \Delta\text{score} + \text{milestones} & \text{(shaped mode)} \\ -1 & \text{(invalid move)} \end{cases} \tag{2}$$

where milestones are one-time bonuses: $+50$ at tile 256, $+100$ at 512, $+200$ at 1024, $+500$ at 2048. Discount factor $\gamma{=}0.99$.

Why 2048 is hard: (1) Sparse, delayed rewards—hundreds of merges precede a single high-value event; (2) Stochastic tile spawns prevent deterministic planning; (3) $\sim 10^{19}$ states make tabular methods infeasible; (4) Episodes last 100–1000+ steps.

III. Architecture & Agents

A. Shared CNN Backbone

Every deep classical agent shares an identical 3-layer CNN feature extractor ϕ_θ to isolate the effect of the learning *algorithm*:

$$\phi_\theta : \mathbb{R}^{16\times 4\times 4} \xrightarrow{\text{Conv}^{2\times 2}_{128}} \xrightarrow{\text{Conv}^{2\times 2}_{128}} \xrightarrow{\text{Conv}^{2\times 2}_{128}} \xrightarrow{\text{FC}_{256}} \mathbb{R}^{256} \tag{3}$$

Each algorithm appends task-specific heads (Q-values, policy logits, value estimates). Total: $\sim$330K parameters per agent.

B. Agent Summary

TABLE I: Seven Agents Benchmarked

Agent	Type	Framework	Params
DQN	Off-policy, value	PyTorch (custom)	330K
PPO	On-policy, A-C	sb3_contrib	330K
A2C	On-policy, A-C	sb3_contrib	330K
QR-DQN	Off-policy, distrib.	sb3_contrib	430K
SAC	Off-policy, max-ent	PyTorch (custom)	330K
LFA	On-policy, linear	NumPy	**108**
GRPO	RLVR (LLM)	Unsloth+TRL	494M

IV. Algorithms

A. DQN (Deep Q-Network)

DQN learns $Q_\theta(s, a)$ via the Bellman target with a frozen target network $\bar{\theta}$:

$$\mathcal{L}_{\text{DQN}} = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_\theta(s, a)\right)^2\right] \tag{4}$$

Algorithm 1 DQN Training Loop

Init Q_θ, $Q_{\bar{\theta}} \leftarrow Q_\theta$, buffer $\mathcal{D}$, $\varepsilon \leftarrow 1.0$
for step $= 1$ **to** N **do**
  $a \leftarrow \begin{cases} \text{random valid action} & \text{w.p. } \varepsilon \\ \arg\max_a Q_\theta(s, a) & \text{otherwise} \end{cases}$
  Execute a; store (s, a, r, s', d) in $\mathcal{D}$
  Sample batch $\mathcal{B} \sim \mathcal{D}$; $y \leftarrow r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$
  Update θ via Adam on SmoothL1(Q_θ, y)
  $\varepsilon \leftarrow \max(0.01,\ 1 - \text{step}/10^5)$
  if step $\bmod 1000 = 0$ **then** $\bar{\theta} \leftarrow \theta$
  end if
end for

Key details: Buffer 100K, batch 64, lr 10^{-4}, grad clip 1.0, 8 parallel envs.

B. PPO (Proximal Policy Optimization)

On-policy actor-critic with clipped surrogate objective:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t\Big[\min\Big(r_t\hat{A}_t,\ \text{clip}(r_t, 1\pm\epsilon)\hat{A}_t\Big)\Big] \tag{5}$$

where $r_t = \pi_\theta(a_t|s_t)/\pi_{\text{old}}(a_t|s_t)$ and $\hat{A}_t$ is GAE(λ=0.95). Uses `MaskablePPO` with invalid actions masked (logits $\to -\infty$). Clip ϵ=0.2, 2048 steps/rollout, 10 epochs/batch, 8 parallel envs.

C. A2C (Advantage Actor-Critic)

Synchronous on-policy with n-step returns:

$$\mathcal{L}_{\text{A2C}} = -\mathbb{E}\Big[\log\pi\hat{A}\Big] + 0.5\,\|R_t - V\|^2 - 0.01\,\mathcal{H}[\pi] \tag{6}$$

n=5 step bootstrap, entropy coefficient 0.01. Uses `MaskableA2C`.

D. QR-DQN (Quantile Regression)

Learns N=50 quantiles of the return distribution $Z(s,a)$:

$$\mathcal{L}_{\text{QR}} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_j\left[\rho^{\kappa}_{\tau_i}(\delta_{ij})\right] \tag{7}$$

Network outputs 4×50=200 values. Actions selected by mean Q-value.

E. Discrete SAC (Soft Actor-Critic)

Maximum-entropy RL with twin Q-networks and automatic temperature:

$$J(\pi) = \sum_t \mathbb{E}_\pi[r_t + \alpha\,\mathcal{H}[\pi(\cdot|s_t)]] \tag{8}$$

$$\mathcal{L}_\pi = \mathbb{E}_s\left[\sum_a \pi(a|s)\Big(\alpha\log\pi(a|s) - \min_k Q_{\psi_k}(s,a)\Big)\right] \tag{9}$$

Target entropy $\bar{\mathcal{H}}$=0.5 $\log|\mathcal{A}|$, Polyak τ=0.005.

F. LFA (Linear Function Approximation)

Semi-gradient SARSA with a hand-crafted 27-dim feature vector $\phi(s)$:

$$\hat{q}(s,a) = \mathbf{w}_a^\top\phi(s), \quad \mathbf{w}\in\mathbb{R}^{4\times 27}\ (108\ \text{params!}) \tag{10}$$

Features: 16 log-tile values, empty ratio, merge potential, 4 monotonicity scores, corner bonus, 3 distribution stats. Update: $\mathbf{w}_a \leftarrow \mathbf{w}_a + \alpha\delta\phi(s)$ with α=5×10^{-4}.

G. GRPO (LLM–RLVR)

Qwen2.5-0.5B-Instruct with QLoRA 4-bit (r=16, α=16), ∼5.2M trainable parameters (1.05%). For each prompt q, generate G=4 completions and normalize advantages within the group:

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \epsilon}, \quad \mu_G = \tfrac{1}{G}\sum_j r_j \tag{11}$$

$$\mathcal{L}_{\text{GRPO}} = \mathbb{E}_q\left[\frac{1}{G}\sum_i\frac{1}{|o_i|}\sum_t\Big(L_t^{\text{clip}} - \beta D_{\text{KL}}\Big)\right] \tag{12}$$

TABLE II: GRPO Reward Components

Component	Value	Description
Format r_f	+0.5	Valid `<think>`/`<answer>` XML tags
Direction r_d	+0.5	Answer ∈ {UP,DOWN,LEFT,RIGHT}
Game r_g	$+1/-2$	Valid move / no-op penalty
Score r_s	$\Delta s/1024$	Normalized merge score
Quality r_q	0→0.5	Strategic vocabulary in `<think>`

Multi-component verifiable reward:
Staged scheduling: Training all rewards simultaneously causes reward hacking (model maximizes length, ignores format). Successful config: Stage 1 = format + direction only, β=0.04, lr 10^{-6}.

V. Results

All classical agents trained for 5M steps with shaped rewards on a single NVIDIA RTX 4060 GPU. DQN uses 8 parallel envs (∼20M total interactions). GRPO trained for 300 steps on 500 board-state prompts with G=4.

A. Performance Comparison

TABLE III: Agent Performance After 5M Training Steps

Agent	Avg Score	Max Score	Max Tile	512+ Rate	1024+ Rate	Params
DQN	**7,744**	**39,314**	**2048**	**76.0%**	**31.0%**	330K
LFA	2,456	11,436	1024	6.0%	<1%	**108**
A2C	1,944	9,640	1024	6.0%	<1%	330K
GRPO	1,816	1,816	128	—	—	494M
SAC*	1,131	5,566	512	<1%	—	330K
QR-DQN	995	5,176	512	<1%	—	430K
PPO	962	4,900	512	<1%	—	330K

*SAC trained ∼4M steps (in progress).

B. Scaling: 1M → 5M Steps

TABLE IV: Sample Efficiency: Improvement from 1M to 5M Steps

Agent	Avg@1M	Avg@5M	Δ	Verdict
DQN	2,825	7,744	**+174%**	Strong scaling
A2C	1,244	1,944	+56%	Modest
SAC	∼700	1,131	+62%	Slow but positive
LFA	2,299	2,456	+7%	Plateaued
QR-DQN	1,090	995	−9%	Degrading
PPO	1,099	962	−12%	Degrading

C. Hunt Mode (Unlimited Attempts)

TABLE V: Hunt-2048: Best Episode with Stochastic Exploration

Agent	Best Tile	Best Score	Strategy
DQN	**2048** ✓	**21,572**	ε=0.05
LFA	1024	12,300	95% greedy
A2C	512	7,844	Dist. sampling
PPO	512	5,860	Dist. sampling
SAC	256	3,648	Prob. sampling
QR-DQN	16	228	ε=0.05

DQN is the **only agent to reach tile 2048**. Adding 5% random exploration breaks deterministic policy cycles at high-tile states where Q-values are poorly estimated.

D. Learning Curves

(a) DQN (5M steps) — avg 7,744

(b) A2C (5M steps) — avg 1,944

(c) LFA (5M steps) — avg 2,456

(d) PPO (5M steps) — avg 962

Fig. 1: Training curves (5M steps, shaped reward). DQN shows strong upward trajectory. PPO flatlines near random baseline.

(a) SAC (∼4M steps) — avg 1,131

(b) QR-DQN (5M steps) — avg 995

Fig. 2: SAC shows no learning trend despite off-policy replay. QR-DQN's moves/episode explode (invalid-move loops).

VI. Analysis & Discussion

A. Off-Policy Replay Is Decisive

DQN's 100K-sample replay buffer lets it revisit rare high-scoring trajectories thousands of times, amplifying sparse reward signals. At 5M steps DQN averages 7,744—a **3× lead** over the next-best agent. On-policy methods (PPO, A2C) discard each trajectory after a single update, creating a chicken-and-egg problem: informative data requires a good policy, which requires informative data.

B. Feature Engineering vs. Deep Learning

LFA with **only 108 parameters** outperforms PPO (330K), QR-DQN (430K), SAC (330K), and the 494M-parameter LLM. Its hand-crafted monotonicity and corner features provide a stronger inductive bias than CNNs learning from scratch. However, LFA's linear capacity limits it below DQN—it cannot capture the complex tile interactions for the $1024 \rightarrow 2048$ merge.

C. Scaling Reveals Divergent Trajectories

Three distinct patterns emerge from 1M → 5M scaling (Table IV):

- **Strong scaling** (DQN, +174%): Off-policy replay compounds with more diverse data.
- **Plateau** (LFA, +7%): Linear model hits its representational ceiling.
- **Degradation** (PPO −12%, QR-DQN −9%): More training → worse policy. PPO wastes probability mass on a fixed suboptimal policy; QR-DQN develops invalid-move loops (moves/episode explode from 200 to 1000+).

D. GRPO: Format Before Strategy

The LLM achieved 30–44% format compliance (`<think>`/`<answer>`) after 300 steps with format-only rewards. Key failure modes:

- **Reward hacking:** Length reward caused collapse—model generated filler text to maximize word count.
- **Gradient instability:** Grad norms exceeded 17,000 at step 297; KL spikes destabilized training.
- **Solution:** Staged rewards (format → direction → game), β=0.04 KL penalty, lr 10^{-6}, gradient clipping 0.5.

E. Inference Latency

TABLE VI: Inference Cost Per Move

Agent	ms/move	Hardware	Params
LFA	<0.1	CPU	108
PPO, A2C, SAC	<1	GPU	330K
DQN	3.6	GPU	330K
QR-DQN	4.2	GPU	430K
LLM (GRPO)	**1,500**	GPU	494M

The LLM is ∼400× slower but provides *interpretable chain-of-thought reasoning*—each move is accompanied by a natural-language explanation.

VII. Conclusion & Future Work

Key findings:

1) **DQN dominates** (avg 7,744, tile 2048 in hunt mode) thanks to off-policy experience replay in this sparse-reward domain.
2) **LFA (108 params) beats all deep on-policy methods**, demonstrating domain-specific features > large neural networks.
3) **On-policy methods fail to scale**: PPO and QR-DQN *degrade* with 5× more training.
4) **GRPO requires staged rewards**: simultaneous multi-reward training causes collapse.
5) **Exploration is essential**: 5% ε-noise breaks deterministic policy cycles at high-tile states.

Future work:

- Complete SAC 5M training (∼1M steps remaining)

- Staged GRPO: Stage 2 (+ game reward), Stage 3 (all rewards)
- Scaling curve evaluation at checkpoints 50K→5M
- Multi-seed experiments (3 seeds $\times$ 7 agents) for error bars
- Hybrid LLM-MCTS for interpretability + search efficiency

Acknowledgment

CS 5180 Reinforcement Learning, Prof. Robert Platt, Khoury College, Northeastern University. All experiments: NVIDIA RTX 4060 (8 GB VRAM), CUDA 12.6.

References

[1] D. Guo *et al.*, "DeepSeek-R1," *Nature*, vol. 645, 2025.
[2] Y. Wen *et al.*, "RLVR Implicitly Incentivizes Reasoning," *arXiv:2506.14245*, 2025.
[3] P. Saligram *et al.*, "2048: RL in Delayed Reward," *arXiv:2507.05465*, 2025.
[4] J. Schulman *et al.*, "PPO Algorithms," *arXiv:1707.06347*, 2017.
[5] W. Dabney *et al.*, "Distributional RL with QR," *AAAI*, 2018.
[6] T. Haarnoja *et al.*, "Soft Actor-Critic," *ICML*, 2018.
[7] L. Hu *et al.*, "lmgame-Bench," *arXiv:2505.15146*, 2025.